Empower Nyamagabe: Technical Report

Rurangwa Mika, BSc Software Engineering, Mission Capstone, October 2026
Repository: https://github.com/mikarurangwa/empower_nyamagabe

1. Scope Alignment and Feature Validation

The table compares the approved proposal (objectives, functional requirements FR1 to FR8 and non-functional requirements NFR1 to NFR6) with what is implemented.

Planned Feature	Intended Purpose	Current Status	Evidence
Family selection process (Objective a)	Fair, transparent choice of vulnerable families	Completed	Ubudehe category, two-person review, written rejection reasons; utils/familyRules.js
FR1 Donor account with phone or email	Donors without email can still register	Completed	services/AuthService.js; tests/integration/auth.test.js
FR2 View verified family profiles	Donors only see approved, real needs	Completed	models/Family.js, utils/ranking.js; Figure 1
FR3 Donate through Flutterwave	Pay by Mobile Money or card without a bank account	Completed (test mode, as planned)	services/DonationService.js, services/payment/FlutterwaveGateway.js
FR4 Link donation to exact need	Donor knows what the money paid for	Completed	beneficiary_id and purpose on every donation; receipt page
FR5 Leader submits a family	Use community knowledge to find households	Completed	services/FamilyService.js; leader dashboard
FR6 Approve or reject a family	A second person checks every family	Completed	checkCanDecide() in utils/familyRules.js; verification.test.js
FR7 Donation history and trace	Proof of where each gift went	Completed	My donations tab, receipt and PDF download; Figure 1
FR8 Admin reports	Oversight of donations and families	Completed	services/ReportService.js, CSV and Excel export
NFR1 Usability	Usable with low digital literacy	Completed	One simple dashboard per role
NFR2 95% availability	Reliable online access	Partial	Health check endpoint ready; hosting not live yet
NFR3 Security and privacy	Protect families and donors	Completed	bcrypt, JWT roles, rate limiting; public data limited to safe fields
NFR4 Confirmation within 30 seconds	Donor knows quickly that payment worked	Completed	Payment checked on return and by webhook; status re-checked every 4 s
NFR5 Works on low-cost phones	Most users have basic smartphones	Completed	Home page 132 KB on a phone; smooth on emulated low-end phone (section 3)
NFR6 Traceability	Trace donor to family to purpose	Completed	Donation trace record and audit log (models/AuditLog.js)
Testing with families and donors (Objective d)	Measure trust and usability with real users	Partial	7 participants tested so far, out of the 50 planned

Scope changes. The build follows the proposal's stack and five modules (Donor, Family, Verification, Donation, Admin). Three additions were made, each supporting an existing requirement: families are ranked so the most urgent appear first (FR2), every decision and payment is written to an audit log (NFR6), and payments are confirmed through both the return redirect and the Flutterwave webhook (NFR4). Payments stay in Flutterwave test mode, as the proposal planned. Hosting (NFR2) and the remaining participant testing (Objective d) are still in progress.

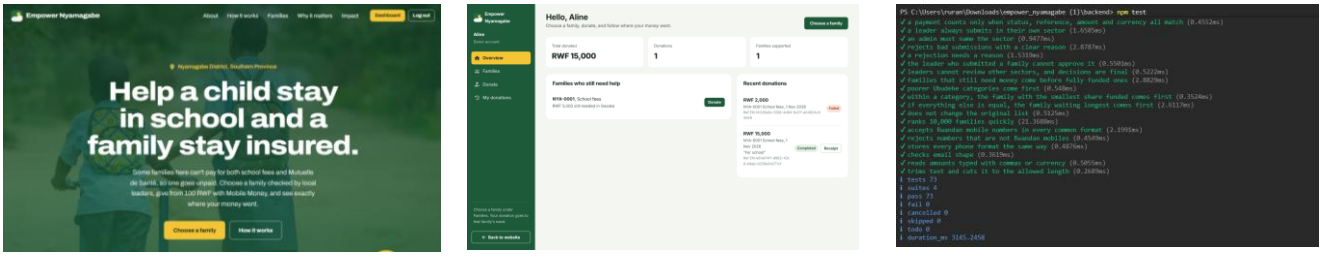


Figure 1. Home page with verified families, donor dashboard, and test results (73 of 73 passing).

2. Technical Functionality

The system has three layers: a front end in HTML, CSS and JavaScript; a REST API in Node.js and Express; and a SQLite database. Payments go through Flutterwave's hosted checkout, so the platform never handles card or Mobile Money details. In the API, each request passes through a route, a controller, a service (business logic) and a model (SQL).

Authentication. Passwords are hashed with bcrypt. Login returns a JWT holding the user's role, checked by middleware on protected routes. Phone numbers are normalised to one format to prevent duplicate accounts, and login attempts are rate-limited. There is no default admin password: the first admin is created from the settings file, and in production the server will not start without its secret settings.

Family verification. A leader submits a household with its Ubudehe category, need and notes. It stays pending and hidden until a second leader in the same sector or the admin approves it. A leader cannot approve a family they submitted, and rejections need a reason.

Donations. The API checks the family is approved and the amount does not exceed what it still needs, saves a pending donation, and gets a checkout link from Flutterwave. On return, the API asks Flutterwave to confirm the payment and marks it completed only if the status, reference, amount and currency all match. Flutterwave's webhook gives a second confirmation path; if no webhook secret is set, webhooks are ignored and a warning is shown at start-up, so payments are then confirmed only when the donor returns from checkout.

Reports and privacy. Donors get receipts as a page and PDF. The admin sees totals, exports CSV and Excel files, and reads the audit log. Public pages only show a family code, sector, Ubudehe category and need.

3. Testing Approaches and Results

Approach	How it was done	Result	Fix or improvement
Unit testing	36 tests on business rules, validators, ranking, classes and admin set-up (Node test runner)	36/36 pass	Rules moved into pure functions so they can be tested alone
Integration testing	37 tests calling the real API over HTTP with an in-memory database and a fake payment gateway	37/37 pass	Found that a late "cancelled" message could undo a completed donation; fixed
Functional testing	Browser run of 14 user steps (login, donate, submit and review a family, sign-up) plus manual testing	All pass	Found the Add leader form crashing (duplicate element ID); fixed
Performance testing	Load test: 100 families, 5,000 donations, 20 parallel connections	See Table 2	Database indexes made the busiest pages 6 to 8 times faster
Device testing	Chromium emulating desktop, mid-range phone (4x slower CPU, Fast 3G) and low-end phone (6x, Slow 3G)	See Table 3	Replaced the full icon library with a 15 KB icon file

Table 1. Testing approaches.

Examples of input testing: a donation below 100 RWF or above the family's remaining need is rejected with a clear message; a payment Flutterwave reports with a different amount or currency is marked failed; a leader approving their own family gets a 403 error; invalid phone numbers, short passwords, broken JSON and missing tokens all return clear 400 or 401 responses.

Endpoint	Without indexes	With indexes	Median response
GET /api/families/verified	64 req/s	528 req/s	300 ms to 36 ms
GET /api/public/impact	37 req/s	239 req/s	554 ms to 81 ms

Table 2. Load test results (2-core Linux machine, zero errors).

Device profile	Home page size	Load time	Scrolling
Desktop	231 KB	0.1 s	60 fps
Mid-range phone, Fast 3G	132 KB	1.1 s	60 fps
Low-end phone, Slow 3G	147 KB	3.2 s	60 fps

Table 3. Front-end performance by device (Chromium).

Browser compatibility: tested on Chrome, Edge, Firefox, Safari and Android.

4. Code Quality Practices

Practice	How it is applied
Encapsulation	Database and FlutterwaveGateway keep the connection and secret key in private fields.
Abstraction	PaymentGateway, Exporter and BaseModel define what subclasses must provide and cannot be used directly.
Inheritance	Six models extend BaseModel; four error classes extend AppError with their HTTP status.
Polymorphism	DonationService works with any PaymentGateway (Flutterwave or the test fake); ExportService calls the same send() on CSV and Excel exporters.
Separation of concerns	Routes, controllers, services, models and utilities each have one job.
Dependency injection	Services receive their models and gateway through constructors, wired in services/index.js.

Algorithms. Each problem was matched to the simplest approach that is correct and explainable to leaders and donors.

Problem	Approach	Why this approach	Cost
Show the most urgent families first	Multi-key comparison sort: open need, Ubudehe category, share funded, waiting time	Each rule can be explained to a leader; a weighted score would hide the trade-offs. Precomputing keys was tested and was slower	O(n log n); 10,000 families in about 3 ms
Know how much a family still needs	Computed from completed donations with an indexed SQL subquery	Never stores a running total that could drift out of sync with the donations	Index lookup instead of a full scan; 6 to 8 times faster
Confirm a payment is real	Four-field match with Flutterwave, and only pending donations can change state	Redirect links can be faked, and late or repeated messages must not undo a payment	O(1) per payment
Stop duplicate accounts	Normalise phone numbers to one format before the uniqueness check	0781234567 and +250781234567 are the same person	O(1)
Safe password resets	Random token sent by email; only its SHA-256 hash stored, one-hour expiry, single use	A leaked database cannot be used to reset passwords	O(1)

Maintainability. The code follows a standard Express folder layout with clear naming, short comments, numbered database migrations, settings in one config file and one central error handler. The frontend has one script per dashboard (donor, leader, admin) plus shared helpers, none over 180 lines. The README covers setup, deployment, the API and testing.